Chapter 29 | Exercise solutions with Python

Scientific Sequences and Biological Structure

This notebook solves Exercises 5–8. It checks normalized composition and pair features, masks special tokens in residue pooling, translates a contact map fixture, and works through class-wise F1 scores.

The arrays and labels are small enough to check by hand. The written solutions cover Exercises 1–4 and 9–11.

To run the notebook, open it inside a copy of `transformers-book` and select a Python kernel with the packages in `requirements.txt`. Run the cells in order. The exercises run on CPU and include their saved outputs.

```
from pathlib import Path
import sys

import numpy as np

working_directory = Path.cwd()
for repo_root in (working_directory, *working_directory.parents):
    model_path = (
        repo_root / "examples" / "ch29_protein"
    )
    if model_path.is_dir():
        sys.path.insert(0, str(repo_root))
        break
else:
    raise FileNotFoundError(
        "Open this notebook inside transformers-book."
    )
```

5. Verify the pair representation

A composition vector gives the fraction of residues that are each letter. In both `CAAC` and `CCAA`, two of the four residues are A and two are C. The A and C entries are each $2/4 = 1/2$. Every other entry is zero.

A dipeptide is a pair of neighboring residues read in sequence order. Move a two-residue window one position at a time. In `CAAC` it gives `CA`, `AA`, and `AC`. In `CCAA` it gives `CC`, `CA`, and `AA`. The pair matrix stores each frequency at the row for the first letter and the column for the second. Each listed pair occurs once among the three windows, so it has frequency $1/3$. All other pair entries are zero.

The expected arrays below come from those counts. The checks compare every entry, including the zeros, and verify that both totals are one for each four-residue sequence.

```
from examples.ch29_protein.model import (
    AMINO_ACIDS,
    dipeptide_frequencies,
    residue_composition,
)

alphabet_size = len(AMINO_ACIDS)
a_index = AMINO_ACIDS.index("A")
c_index = AMINO_ACIDS.index("C")
expected_composition = np.zeros(alphabet_size)
expected_composition[[a_index, c_index]] = 1 / 2

pairs_by_sequence = {
    "CAAC": ("CA", "AA", "AC"),
    "CCAA": ("CC", "CA", "AA"),
}
for sequence, pairs in pairs_by_sequence.items():
    composition = residue_composition(sequence)
    frequencies = dipeptide_frequencies(sequence)

    expected_frequencies = np.zeros((alphabet_size, alphabet_size))
    for first, second in pairs:
        row = AMINO_ACIDS.index(first)
        column = AMINO_ACIDS.index(second)
        expected_frequencies[row, column] = 1 / 3

    np.testing.assert_allclose(composition, expected_composition)
    np.testing.assert_allclose(frequencies, expected_frequencies)
    np.testing.assert_allclose(composition.sum(), 1)
    np.testing.assert_allclose(frequencies.sum(), 1)

    print(f"{sequence}")
    print(f"  A={composition[a_index]:g}, C={composition[c_index]:g}")
    for first, second in ("AA", "AC", "CA", "CC"):
        row = AMINO_ACIDS.index(first)
        column = AMINO_ACIDS.index(second)
        print(f"  {first}{second}={frequencies[row, column]:.6f}")
    print(f"  Composition total = {composition.sum():g}")
    print(f"  Dipeptide total   = {frequencies.sum():g}")
```

```
OUTPUT
CAAC
  A=0.5, C=0.5
  AA=0.333333
  AC=0.333333
  CA=0.333333
  CC=0.000000
  Composition total = 1
  Dipeptide total   = 1
CCAA
  A=0.5, C=0.5
  AA=0.333333
  AC=0.000000
  CA=0.333333
  CC=0.333333
  Composition total = 1
  Dipeptide total   = 1
```

The sequence `A` contains one residue, so its composition assigns all the frequency to A. Its composition total is still one. It contains zero adjacent pairs, so there is no positive pair count to divide by. The function returns a matrix of zeros in this case, with total zero.

```
composition = residue_composition("A")
frequencies = dipeptide_frequencies("A")

expected_composition = np.zeros(alphabet_size)
expected_composition[a_index] = 1
np.testing.assert_array_equal(composition, expected_composition)
np.testing.assert_array_equal(
    frequencies, np.zeros((alphabet_size, alphabet_size))
)
np.testing.assert_allclose(composition.sum(), 1)
np.testing.assert_allclose(frequencies.sum(), 0)

print(f"A composition entry = {composition[a_index]:g}")
print(f"Composition total   = {composition.sum():g}")
print(f"Dipeptide total     = {frequencies.sum():g}")
```

```
OUTPUT
A composition entry = 1
Composition total   = 1
Dipeptide total     = 0
```

6. Change only padding

The chapter's pooling figure shows five vectors in token order. The beginning marker (BOS) has $(9, 9)$, residue A has $(1, 0)$, residue C has $(1, 2)$, the ending marker (EOS) has $(-9, -9)$, and padding (PAD) has $(0, 0)$. The mask has one Boolean value per row. Only the two residue rows have a value of `True`.

Mean pooling averages each coordinate over the selected rows.

$$\frac{(1,0)+(1,2)}{2} = (1,1).$$

Copy the array, replace the last row with $(7, 9)$, and pass the same mask both times. The two selected rows and their count stay the same. Both runs should therefore return $(1, 1)$.

```
from examples.ch29_protein.model import mean_pool_residues

embeddings = np.array([
    [9, 9],    # BOS
    [1, 0],    # A
    [1, 2],    # C
    [-9, -9],  # EOS
    [0, 0],    # PAD
])
residue_mask = np.array([False, True, True, False, False])

pooled = mean_pool_residues(embeddings, residue_mask)
changed_embeddings = embeddings.copy()
changed_embeddings[-1] = [7, 9]
pooled_after = mean_pool_residues(changed_embeddings, residue_mask)

np.testing.assert_array_equal(pooled, [1, 1])
np.testing.assert_array_equal(pooled_after, pooled)

print(f"Pooled with padding (0, 0) = {pooled}")
print(f"Pooled with padding (7, 9) = {pooled_after}")
```

```
OUTPUT
Pooled with padding (0, 0) = [1. 1.]
Pooled with padding (7, 9) = [1. 1.]
```

Both calls return the same two coordinates. The function selects rows using the mask before taking the mean. The padding vector contributes neither to the coordinate sums nor to the number of rows averaged.

7. Translate the coordinates

Use the four points from the chapter's contact-map figure. In chain order they are $(0, 0, 0)$, $(3, 0, 0)$, $(0, 4, 0)$, and $(8, 0, 0)$. A pair is a contact when its Euclidean distance is at most 5 and its residue numbers differ by at least 2.

Using one-based residue numbers, only three unordered pairs pass the sequence-separation rule.

Pair	Index separation	Distance	Contact
$(1, 3)$	2	4	Yes
$(1, 4)$	3	8	No
$(2, 4)$	2	5	Yes

For each of these pairs, just one coordinate differs, so the distance is the size of that difference. Pair $(2, 4)$ qualifies at both cutoffs. Pair $(1, 2)$ is only 3 units apart, but its index separation is 1, so its entry is false. The diagonal has separation 0. Each accepted pair contributes two true entries, one in each half of the symmetric matrix.

```
from examples.ch29_protein.model import contact_map

coordinates = np.array(
    [[0, 0, 0], [3, 0, 0], [0, 4, 0], [8, 0, 0]]
)
threshold = 5
separation = 2
original_map = contact_map(coordinates, threshold, separation)

expected_map = np.array(
    [[0, 0, 1, 0],
     [0, 0, 0, 1],
     [1, 0, 0, 0],
     [0, 1, 0, 0]],
    dtype=bool,
)
np.testing.assert_array_equal(original_map, expected_map)
print("Original map (1 = contact)")
print(original_map.astype(int))
```

```
OUTPUT
Original map (1 = contact)
[[0 0 1 0]
 [0 0 0 1]
 [1 0 0 0]
 [0 1 0 0]]
```

A translation adds the same vector to every point. If $\mathbf{p}_i$ is the coordinate of residue i and $\mathbf{t} = (10, -3, 2)$, then

$$(\mathbf{p}_i + \mathbf{t}) - (\mathbf{p}_j + \mathbf{t}) = \mathbf{p}_i - \mathbf{p}_j.$$

Taking the length of that difference gives the same distance as before. The chain order stays fixed, so the index separation also stays fixed. Since each entry depends only on these two quantities, a shared translation preserves the whole map.

```
translation = np.array([10, -3, 2])
shifted_coordinates = coordinates + translation
shifted_map = contact_map(shifted_coordinates, threshold, separation)

np.testing.assert_array_equal(shifted_map, original_map)
print("Translated map (1 = contact)")
print(shifted_map.astype(int))
```

```
OUTPUT
Translated map (1 = contact)
[[0 0 1 0]
 [0 0 0 1]
 [1 0 0 0]
 [0 1 0 0]]
```

8. Measure two classes

The first two proteins are correctly predicted as A. The third is an A predicted as B, and the fourth is the only true B. We score each class separately. For the class being scored, a true positive (*TP*) has that true label and is predicted to have it. A false positive (*FP*) receives that prediction in error. A false negative (*FN*) has that true label but is predicted to belong to the other class.

For each class, compute $F_1 = 2TP/(2TP + FP + FN)$. Both classes have a positive denominator in both runs. The support is the number of proteins with that true label, which equals $TP + FN$. Macro-F1 is the mean of the two class scores, giving A and B equal weight even though A has three examples and B has one.

```
true_labels = np.array(["A", "A", "A", "B"])
original_predictions = np.array(["A", "A", "B", "B"])
changed_predictions = original_predictions.copy()
changed_predictions[-1] = "A"
classes = ("A", "B")


def class_scores(true_labels, predictions, label):
    has_label = true_labels == label
    predicted_label = predictions == label
    tp = int(np.count_nonzero(has_label & predicted_label))
    fp = int(np.count_nonzero(~has_label & predicted_label))
    fn = int(np.count_nonzero(has_label & ~predicted_label))
    f1 = 2 * tp / (2 * tp + fp + fn)
    return tp, fp, fn, f1
```

Before the change, protein 3 contributes a false negative for A and a false positive for B. This gives A an F1 of 4/5 and B an F1 of 2/3, for a macro-F1 of 11/15.

After the change, protein 4 also becomes an error. It contributes a false positive for A and a false negative for B. A keeps its two true positives and now scores $4/6 = 2/3$. B loses its only true positive and scores 0. The new macro-F1 is 1/3.

```
comparisons = [
    ("Original predictions", original_predictions),
    ("Changed final prediction", changed_predictions),
]
expected_counts = [[(2, 0, 1), (1, 1, 0)], [(2, 1, 1), (0, 1, 1)]]
expected_f1 = [[4 / 5, 2 / 3], [2 / 3, 0]]
expected_macro = [11 / 15, 1 / 3]

for index, (title, predictions) in enumerate(comparisons):
    print(title)
    print("Class  Support  TP  FP  FN      F1")
    scores = []
    for label, counts in zip(classes, expected_counts[index]):
        tp, fp, fn, f1 = class_scores(true_labels, predictions, label)
        assert (tp, fp, fn) == counts
        scores.append(f1)
        print(f"{label:<5} {tp + fn:>7} {tp:>3} {fp:>3} {fn:>3}"
              f" {f1:>7.4f}")

    macro_f1 = float(np.mean(scores))
    np.testing.assert_allclose(scores, expected_f1[index])
    np.testing.assert_allclose(macro_f1, expected_macro[index])
    print(f"Macro-F1 = {macro_f1:.4f}\n")
```

OUTPUT

```
Original predictions
Class  Support  TP  FP  FN      F1
A           3   2   0   1  0.8000
B           1   1   1   0  0.6667
Macro-F1 = 0.7333

Changed final prediction
Class  Support  TP  FP  FN      F1
A           3   2   1   1  0.6667
B           1   0   1   1  0.0000
Macro-F1 = 0.3333
```